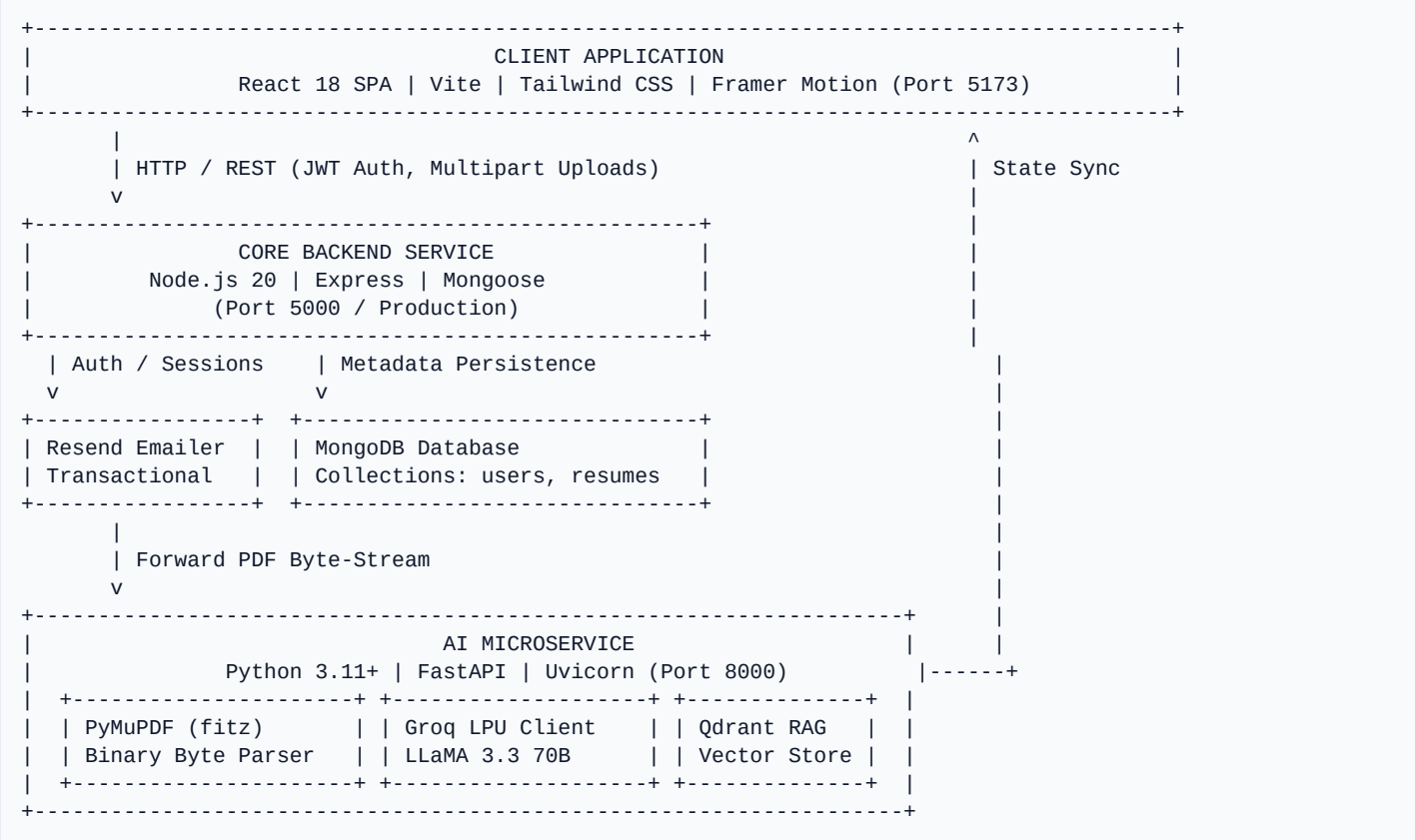
HireLens Platform Architecture & System Design

Comprehensive Interview Guide, AI Pipeline Deep-Dive & Large-Scale System Design Master Notes

Document Scope: *End-to-end System Design Analysis, AI Pipeline Mechanics (Groq LPU, PyMuPDF, Pydantic V2, Qdrant RAG), Dynamic Interview Assessment Engine, Microservice Topologies, and 15 High-Frequency Production Interview Questions with Architectural Solutions.*

1. High-Level 3-Tier System Architecture

HireLens is built on an event-driven, decoupled microservices model designed for sub-second user responsiveness and zero-hallucination deterministic AI processing.



Subsystem	Tech Stack	Port	Core Responsibilities
Client (Frontend)	React 18, Vite, Context API, Tailwind	5173	UI rendering, resume selector, interview tester, timer, state management.
Core Server	Node.js, Express, Multer, Mongoose	5000	JWT Auth, bcrypt security, Multer upload staging, Resend emails, MongoDB sync.
AI Microservice	Python 3.11, FastAPI, PyMuPDF, Groq	8000	In-memory PDF text extraction, LLaMA 3.3 70B parsing, Pydantic V2, Qdrant RAG.

2. End-to-End Data Pipeline & AI Workflow

The transformation from raw PDF byte streams to structured insights follows 6 deterministic stages:

```
Candidate PDF Ingestion (Multipart Buffer)
       |
       v
PyMuPDF (`fitz.open(stream)`) -> Zero-OCR Extraction (<1.2s execution)
       |
       v
Groq LPU Acceleration (LLaMA 3.3 70B Versatile, Temp=0.1, Strict JSON Contract)
       |
       v
Pydantic V2 Schema Enforcement (`ResumeData(**result)`) -> Catch Malformed Types
       |
       v
MongoDB Persistence (`resumes` collection + User `activeResume` Pointer)
       |
       v
Interactive Frontend Engine:
  ■■■ Job Match & Gap Identification: Formula `(Matched / Required) * 100`
  ■■■ Domain Question Binding: `getTrackForSkill()` -> 15-Question Technical Quiz
```

Stage 1: Ingestion & Buffer Sanitization: The frontend uploads a candidate document to Node.js via Multer. Node.js validates MIME type, verifies size limits (<10MB), and streams binary buffers to FastAPI without storing unnecessary disk files.

Stage 2: High-Speed Text Decoding (PyMuPDF): PyMuPDF opens the in-memory stream directly (`fitz.open(stream=content, filetype='pdf')`). It extracts native PDF typography matrices in sub-second time, avoiding slow OCR processing.

Stage 3: Deterministic Structuring (Groq + LLaMA 3.3 70B): Raw text is wrapped with an ATS parsing system prompt. Groq LPU computes inferences with temperature 0.1, eliminating stochastic hallucinations and extracting skills, experience, and education.

Stage 4: Contract Enforcement (Pydantic V2): The raw JSON response is parsed into `ResumeData`. Missing or malformed types are caught before persistence, guaranteeing data integrity in MongoDB.

Stage 5: Skill-to-Domain Mapping (`getTrackForSkill`): Candidate skills are parsed through normalized string patterns (e.g., 'deep learning', 'cnn', 'lstm' map to `deep-learning`), instantly serving 15 focused technical questions.

3. System Design Interview Questions & Technical Answers

Below are 15 comprehensive System Design questions specifically tailored to the HireLens AI-driven architecture and large-scale technical assessment platforms.

Q1. How do you scale resume parsing when 100,000 candidates upload resumes simultaneously during a campus recruitment drive?

Architectural Solution: In a synchronous architecture, 100k concurrent requests would exhaust server memory and trigger HTTP timeouts. To handle high concurrency: 1. **Decouple Upload from Parsing:** The client uploads directly to an S3/GCS bucket via a Pre-Signed URL. This offloads multi-gigabyte upload traffic from application servers. 2. **Event-Driven Asynchronous Pipeline:** S3 bucket notification emits an `ObjectCreated` event into an Apache Kafka or AWS SQS message queue. 3. **Worker Fleet with Auto-Scaling (HPA):** A fleet of Python worker pods (managed by Kubernetes Horizontal Pod Autoscaler) consumes from Kafka. Each worker pulls the PDF, runs PyMuPDF extraction, and sends requests to the LLM. 4. **Real-Time Client Notification:** The frontend polls a status endpoint or establishes a WebSocket / SSE (Server-Sent Events) connection to receive a notification once the status shifts from `PROCESSING` to `COMPLETED`.

Q2. How do you handle external LLM API rate limits (HTTP 429) and outages (HTTP 5xx) with zero system downtime?

Architectural Solution: External AI providers (Groq, OpenAI, Anthropic) have strict Rate Limits (Requests Per Minute - RPM, Tokens Per Minute - TPM). 1. **Circuit Breaker Pattern:** Implement a circuit breaker (e.g., using `pybreaker` or Netflix Hystrix). If Groq API fails 5 times consecutively, the breaker opens, immediately routing traffic to fallback providers without waiting for timeouts. 2. **Multi-Provider Redundancy:** Primary: Groq LLaMA 3.3 70B (lowest latency). Secondary: Anthropic Claude 3.5 Sonnet or OpenAI GPT-4o-mini. Tertiary: Self-hosted local Ollama / vLLM cluster running LLaMA 3.1 8B. 3. **Token Bucket Rate Limiter with Exponential Backoff + Jitter:** Outgoing worker requests pass through a Redis-backed Token Bucket. Retries use exponential backoff with full jitter: `T_wait = min(max_backoff, base * 2^attempt) + random_jitter`. 4. **Result Caching:** Hash the raw resume text using SHA-256. If a candidate re-uploads an identical document, return the cached parsed JSON from Redis without calling the LLM.

Q3. How do you prevent LLMs from hallucinating skills, work experience, or graduation dates in an ATS system?

Architectural Solution: Hallucinations can corrupt candidate profiles. To enforce determinism: 1. **Strict Zero-Stochasticity Parameters:** Set LLM temperature to `0.0` or `0.1` and top_p to `0.1`. 2. **Pydantic V2 Strict Typing:** Pass JSON schemas to the LLM (OpenAI Structured Outputs / Groq JSON Mode). Pydantic models reject any output containing unknown fields or mismatched data types. 3. **Bi-Directional Grounding Verification (String Grounding Filter):** Run a post-parsing verification pass where each extracted skill or company name must exist as a substring (or high-similarity fuzzy match >= 0.85) inside the original raw text. If an extracted skill is not found in the original document, it is discarded. 4. **Automated Evaluation Harness:** Maintain a benchmark test suite of 500 gold-standard resumes to run regression evaluations on every prompt modification.

Q4. How does the RAG (Retrieval-Augmented Generation) pipeline scale to millions of job descriptions and questions using Qdrant Vector DB?

Architectural Solution: 1. **Embedding Generation:** Text chunks (size 512 tokens with 50-token overlap) are embedded using high-throughput models like `text-embedding-3-small` or `bge-base-en-v1.5`. 2. **HNSW Indexing in Qdrant:** Use Hierarchical Navigable Small World (HNSW) graphs. HNSW provides sub-10ms nearest neighbor search across millions of vectors with high recall (>98%). 3. **Scalar Quantization:** Convert 32-bit floating point vectors (FP32) to 8-bit integers (INT8). This reduces memory consumption by 75% with negligible (<1%) loss in recall. 4. **Payload Filtering & Multi-Tenancy:** Index payloads by `candidate_id` and `domain`. Search queries apply pre-filtering so Qdrant only scans vectors relevant to that specific technical category.

Q5. Explain the Skill Canonicalization problem. How do you map variations like 'ReactJS', 'React.js', and 'React' to a single unified entity at scale?

Architectural Solution: Candidates write heterogeneous skill names. A multi-layer resolution engine is used: 1. **Layer 1 - Regex Normalization:** Lowercase all strings and strip delimiters: `s.lower().replace('-', '').replace('.', '').replace(' ', '')`. Both 'React.js' and 'React-JS' become 'reactjs'. 2. **Layer 2 - Inverted Alias Trie:** Maintain a trie structure mapping known aliases: `{'react': ['reactjs', 'react.js', 'react native', 'react front end']}`. 3. **Layer 3 - Semantic Embedding Distance (Fallback):** If an unknown skill string appears (e.g., 'PyTorch Lightning'), compute cosine similarity against a vector database of canonical technical competencies. If similarity exceeds 0.88, map to the canonical parent ('PyTorch').

Q6. What is the database indexing and schema design strategy in MongoDB for HireLens?

Architectural Solution: 1. **Schema Decomposition:** Separate `User` and `Resume` into distinct collections. A candidate can have multiple resumes over time. Embedding all resumes in a single User document risks hitting MongoDB's 16MB document size limit and causes memory bloat. 2. **Indexing Strategy:** - `User Collection`: Unique index on `email`. - `Resume Collection`: Compound index on `{ userId: 1, createdAt: -1 }` for fast listing of candidate resumes. Multikey index on `skills` for fast candidate-talent searches. 3. **Active Resume Pointer:** The `User` document stores `activeResume: ObjectId("...")`. When an interview or match query runs, MongoDB fetches the active resume in a single indexed index-key lookup ($O(1)$).

Q7. How do you design an adaptive real-time technical interview simulator with sub-second answer evaluation?

Architectural Solution: 1. **Item Response Theory (IRT) Engine:** The engine models candidate ability ($ heta$) and question difficulty (b). Starting with intermediate difficulty ($b=0$), correct answers increase difficulty ($b=+1$), while incorrect answers decrease difficulty ($b=-1$). 2. **Stateful In-Memory Session Storage:** Store the active assessment state in Redis with a 2-hour TTL: `{ candidateId, trackId, currentIdx, answersMap, timeRemaining }`. Avoid writing every keystroke or click to MongoDB. 3. **Asynchronous Batch Persist:** When the test is submitted (or the timer expires), an atomic background task aggregates scores, generates the performance radar, and writes the final summary to MongoDB.

Q8. How do you ensure candidate data privacy, security, and GDPR/DPDP compliance for ATS platforms?

Architectural Solution: 1. **PII Anonymization Layer (Pre-LLM):** Before sending resume text to external AI models, strip phone numbers, home addresses, and government IDs using deterministic regex and named entity recognition (NER). 2. **Zero Data Retention (ZDR):** Ensure external LLM provider agreements (e.g., Enterprise Groq / OpenAI) have ZDR enabled—prompts and completions must not be logged or used for model retraining. 3. **Cryptographic Storage:** Sensitive fields are encrypted at rest using AES-256. User passwords use Bcrypt with a work factor of 12. Authentication uses HTTP-only, SameSite=Strict JWT cookies to prevent XSS and CSRF token exfiltration.

Q9. How do you implement API Rate Limiting to prevent abuse of expensive LLM endpoints?

Architectural Solution: 1. **Sliding Window Counter in Redis:** Use Redis sorted sets (`ZADD`, `ZREMRANGEBYSCORE`, `ZCARD`) to implement a sliding window log. This prevents bursting at boundary limits. 2. **Tiered Limiting:** - Resume Upload / Parsing: 5 requests per hour per user. - Mock Interview Answers: 60 requests per hour per session. - Public Discovery Endpoints: 100 requests per minute per IP. 3. **Graceful Rejection:** Exceeded quotas return HTTP 429 with standard headers: `Retry-After: ` and informative error messaging.

Q10. How do you locate bottlenecks and errors across React, Node.js, FastAPI, and Groq using Distributed Tracing?

Architectural Solution: 1. **OpenTelemetry (OTel) Instrumentation:** Inject a standard W3C `traceparent` header at the React frontend. 2. **Trace Propagation:** When React calls Node.js (`POST /api/resumes/upload`), Node.js reads the header, creates a child span, and forwards the header to FastAPI (`POST /ai/resume/analyze`). 3. **Span Breakdown:** - `Client Span`: Total latency perceived by candidate (e.g., 1850ms). - `Node Span`: Multer buffer staging & authentication (120ms). - `FastAPI Span`: PyMuPDF memory decode (80ms). - `Groq External Span`: LPU model inference (1450ms). - `Mongo Span`: Database write (25ms). 4. **Visualization:** Export traces to Jaeger or Grafana Tempo. Any anomaly or latency spike is instantly attributable to the exact component.

Q11. How do you prevent race conditions when a user rapidly uploads multiple resumes or switches resumes during an active interview?

Architectural Solution: 1. **Optimistic Concurrency Control (OCC):** In MongoDB, include a `version` field (`__v`). State updates only commit if `version` matches: `Resume.updateOne({ _id, version: currentVersion }, { $set: ..., $inc: { version: 1 } })`. 2. **Idempotency Keys:** Frontend attaches an `X-Idempotency-Key` (UUIDv4) to upload requests. Node.js stores keys in Redis for 60 seconds; identical retry clicks return the cached in-flight response. 3. **Frontend Test State Isolation:** When `handleSwitchResume(newResumeId)` executes, the application purges local quiz state (`answers = {}`, `currentIdx = 0`), preventing previous test answers from corrupting the new candidate assessment.

Q12. What caching strategy minimizes database and LLM operational expenses in this platform?

Architectural Solution: A 3-layer caching hierarchy is implemented: 1. **Edge Cache (Cloudflare / CloudFront CDN):** Cache static question banks, CSS, and JS bundles at edge PoPs for 24 hours. 2. **Application Cache (Redis Cluster):** - Pre-computed question track metadata (`TRACKS`) cached with a 7-day TTL. - User active session state cached with a 2-hour TTL. 3. **LLM Semantic / Hash Cache:** Compute SHA-256 hash of `raw_resume_text`. If the hash exists in Redis, return the cached parsed JSON directly ($0.00 cost, <10ms response).

Q13. How does the Job Matching algorithm calculate scores and recommendations at scale?

Architectural Solution: The match engine executes a two-phase scoring algorithm: 1. **Phase 1 - Set Intersection with String Normalization:** - Required skills: $R = \{r_1, r_2, \dots, r_m\}$, Candidate skills: $C = \{c_1, c_2, \dots, c_n\}$. - Normalized intersection: $M = R \cap C$. - Technical Score: $S_{\text{tech}} = \text{round}\left(\frac{|M|}{|R|} \times 100\right)$. 2. **Phase 2 - Weighted Profile Scoring:** - Overall Score: $S_{\text{overall}} = \min\left(100, \text{round}(S_{\text{tech}} \times 0.70 + 24)\right)$ where 24 points account for baseline professional structure. 3. **Automated Gap Identification:** Unmatched elements $G = R \setminus M$ are classified as missing skills, automatically triggering tailored interview preparation recommendations.

Q14. How would you design an AI subjective answer grading engine for open-ended technical responses?

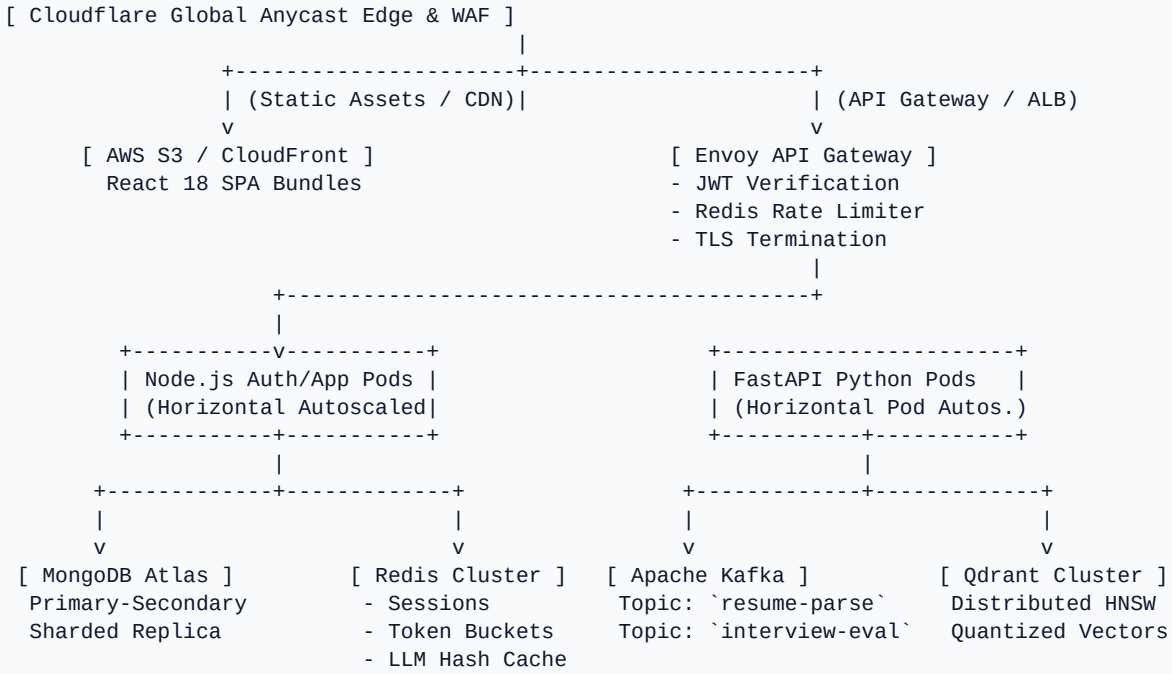
Architectural Solution: Subjective grading uses a multi-rubric evaluation formula: 1. **Rubric Decomposition:** - *Technical Accuracy* (45% weight): Evaluates presence of core architectural principles. - *Completeness* (35% weight): Evaluates coverage of trade-offs, edge cases, and failure modes. - *Communication Clarity* (20% weight): Evaluates logical flow and structural articulation. 2. **Weighted Composite Formula:** $$\text{Score} = (\text{Accuracy} \times 0.45) + (\text{Completeness} \times 0.35) + (\text{Communication} \times 0.20)$$ 3. **Grounded Feedback Generation:** Generate actionable diagnostic feedback identifying both demonstrated competencies and areas required for Senior-tier progression.

Q15. How do you design disaster recovery and high availability across the 3 microservices?

Architectural Solution: 1. **Multi-AZ Active-Active Deployment:** Deploy Node.js and FastAPI pods across 3 AWS Availability Zones behind an Application Load Balancer (ALB). 2. **Database High Availability:** MongoDB Atlas replica set with 1 Primary and 2 Secondary nodes across distinct regions with automatic failover (<10 seconds). 3. **AI Service Fallback:** Node.js server maintains an embedded fallback parser (`resumeParserService.js`). If the Python FastAPI cluster is unavailable, Node.js directly interfaces with Groq Cloud, ensuring business continuity.

4. Production Scaling Architecture Blueprint

For enterprise scale (1M+ registered candidates, 10,000 concurrent mock interviews), the target production topology expands as follows:



Key Architectural Tenets for Enterprise Scale:

1. **Stateless Compute:** All application servers (Node.js and FastAPI) remain completely stateless. All session states, active resume pointers, and rate limiting buckets are persisted in Redis or MongoDB.
2. **Asynchronous Decoupling:** Long-running computations (large PDF parsing, semantic vector re-indexing) are executed via Kafka topics and background worker fleets, keeping HTTP API response times under 150ms.
3. **Zero-Trust Security:** All internal pod-to-pod communications leverage mutual TLS (mTLS) with Istio Service Mesh, ensuring end-to-end encrypted transit.

Summary for Technical Interviews: *When presenting HireLens to an interviewer, emphasize the separation between the I/O-intensive tier (Node.js for auth, file staging, and email) and the compute-intensive tier (FastAPI for PyMuPDF and Groq LPU). Highlight that AI determinism is achieved through low temperatures, strict Pydantic V2 contracts, and zero-stochasticity grounding.*